

TRABAJO FINAL INTEGRADOR (TFI) — FÍSICA I

Simulación de Tiro Parabólico: Comparación entre Modelo Ideal y Real con Resistencia del Aire

- Carrera: Ingeniería en Informática
 - Materia: Física I
 - Nivel: 2do Año
 - Integrantes:
 - Diaz Christopher
 - Donaire Adrián
 - Tecnologías utilizadas: Python 3 (NumPy, SciPy, Matplotlib)
-

1. RESUMEN DEL PROYECTO

Este proyecto consiste en el desarrollo de un simulador computacional en dos dimensiones (2D) para estudiar y comparar el movimiento de un proyectil bajo dos escenarios:

1. Modelo Ideal (en el vacío): No existe rozamiento con el aire. Solo actúa la gravedad. Este caso se resuelve mediante las fórmulas clásicas del tiro parabólico.
2. Modelo Real (con rozamiento del aire): El proyectil experimenta una fuerza de frenado provocada por la resistencia aerodinámica (proporcional al cuadrado de la velocidad). Como las ecuaciones no se pueden resolver de forma directa con una fórmula simple, se utilizan métodos de cálculo numérico por computadora.

El sistema fue programado en Python utilizando Programación Orientada a Objetos (POO). Permite calcular y comparar alcances, alturas, tiempos de vuelo y velocidades de impacto, mostrando los resultados en tablas, gráficos detallados y una animación interactiva con datos en tiempo real.

2. OBJETIVOS

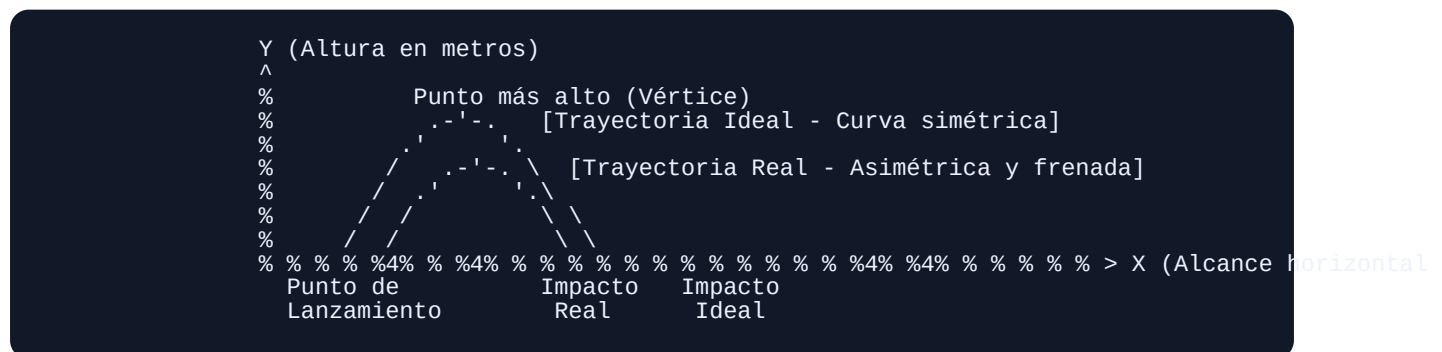
Objetivo General

Modelar, programar y analizar el comportamiento de un proyectil lanzado en un campo gravitatorio constante, comparando la trayectoria ideal contra la trayectoria real afectada por el rozamiento del aire.

Objetivos Específicos

1. Aplicar las leyes de Newton para modelar la fuerza de gravedad y la fuerza de resistencia del aire (arrastre aerodinámico).
2. Implementar algoritmos de integración numérica paso a paso: Runge-Kutta de 4to Orden (RK4) y el Método de Euler.
3. Calcular con precisión el punto de impacto en el suelo y el punto más alto alcanzado (vértice).
4. Generar gráficos comparativos claros de trayectoria, velocidad, aceleración y una animación visual interactiva del disparo.

3. MODELO FÍSICO Y ECUACIONES



3.1. Caso 1: Modelo Ideal (Sin rozamiento con el aire)

En el vacío, la única fuerza que actúa sobre el objeto es su propio peso (gravedad hacia abajo).

- Aceleración horizontal: $a_x = 0$ (la velocidad horizontal no cambia).
- Aceleración vertical: $a_y = -g$ (la gravedad frena la subida y acelera la caída, con $g = 9.81 \text{ m/s}^2$).

Fórmulas de velocidad en cada instante (t):

- Velocidad horizontal: $v_x(t) = v_0 \cdot \cos(\theta)$
- Velocidad vertical: $v_y(t) = v_0 \cdot \sin(\theta) - g \cdot t$

Fórmulas de posición:

- Posición horizontal: $x(t) = x_0 + v_x \cdot t$
- Posición vertical: $y(t) = y_0 + v_{y0} \cdot t - 0.5 \cdot g \cdot t^2$

Resultados principales:

- Tiempo total de vuelo: Tiempo que tarda en volver a tocar el suelo ($y = 0$).
- Altura máxima: Punto donde la velocidad vertical se anula ($v_y = 0$).
- Alcance máximo: Distancia horizontal total recorrida.

3.2. Caso 2: Modelo Real (Con resistencia del aire)

Cuando un objeto se mueve en la atmósfera a velocidades normales o altas, el aire ejerce una fuerza de frenado opuesta a la dirección del movimiento llamada Fuerza de Arrastre (F_d).

¿De qué depende la resistencia del aire?

La fuerza de frenado del aire depende de cuatro factores:

1. Densidad del aire (ρ): Aproximadamente 1.225 kg/m^3 al nivel del mar.
2. Coeficiente de arrastre (C_d): Indica qué tan aerodinámica es la forma del cuerpo (para una esfera lisa es aproximadamente 0.47).
3. Área frontal (A): La superficie frontal del proyectil ($A = \pi \cdot r^2$).
4. Velocidad al cuadrado (v^2): A mayor velocidad, la resistencia crece de manera cuadrática.

Agrupando las constantes en un único factor de resistencia (b):

$$b = 0.5 \cdot \text{Densidad} \cdot \text{Coeficiente_Arrastre} \cdot \text{Área}$$
$$\text{Fuerza_Arrastre} = b \cdot (\text{Velocidad})^2$$

Ecuaciones de movimiento (Segunda Ley de Newton):

Al descomponer las fuerzas en los ejes X e Y:

- Eje horizontal (X): Solo actúa la resistencia del aire frenando el proyectil:

$$\text{Aceleración_X} = - (b / \text{Masa}) \cdot \text{Velocidad_Total} \cdot \text{Velocidad_X}$$

- Eje vertical (Y): Actúan la gravedad hacia abajo y la resistencia del aire opuesta al movimiento vertical:

$$\text{Aceleración}_Y = - \text{Gravedad} - (b / \text{Masa}) * \text{Velocidad}_{\text{Total}} * \text{Velocidad}_Y$$

(Donde la $\text{Velocidad}_{\text{Total}} = \sqrt{(\text{Velocidad}_X^2 + \text{Velocidad}_Y^2)}$)

Explicación clave: En el modelo real, la velocidad horizontal y la vertical están conectadas entre sí a través de la velocidad total. Por esta razón no existe una fórmula directa cerrada para calcular la posición en cualquier instante; es necesario calcular la trayectoria avance por avance utilizando métodos numéricos.

4. MÉTODOS DE CÁLCULO NUMÉRICO

Para conocer la posición y velocidad en cada milisegundo de la simulación, se divide el tiempo en pasos pequeños (por ejemplo, $dt = 0.001$ segundos) y se calculan las variaciones paso a paso.

4.1. Método de Euler (Básico)

Calcula el siguiente punto multiplicando la velocidad y aceleración actuales por el paso de tiempo:

- Nueva posición = Posición actual + Velocidad * dt
- Nueva velocidad = Velocidad actual + Aceleración * dt

Es fácil de entender pero acumula errores a lo largo del tiempo si el paso no es extremadamente pequeño.

4.2. Método de Runge-Kutta de 4to Orden — RK4 (Avanzado y Preciso)

En lugar de tomar solo la aceleración al inicio del intervalo, el método RK4 calcula 4 estimaciones de la pendiente en distintos puntos del intervalo de tiempo y obtiene un promedio ponderado.

- Brinda una precisión extremadamente alta (error prácticamente nulo comparado con el modelo teórico).
- Es el método principal empleado por este simulador.

4.3. Detección exacta del impacto en el suelo

Durante la simulación, el último paso suele quedar ligeramente por debajo del nivel del suelo ($y < 0$). Para no cometer error en el alcance final:

- Se utiliza una interpolación matemática para encontrar con exactitud decimal el instante en el que la altura cruza exactamente el cero ($y = 0$).

- Esto permite obtener el alcance final y la velocidad de impacto exactos.

5. ESTRUCTURA Y DISEÑO DEL PROGRAMA

El código fuente está organizado en 5 módulos independientes para mantener un diseño limpio, modular y fácil de mantener:

[illegible]

Descripción de los Módulos:

1. `modelo.py`: Define la clase `Proyectil` con sus propiedades físicas (masa, radio, área, coeficiente de arrastre) y las funciones de derivadas físicas e integración (RK4 y Euler).
2. `simulador.py`: Contiene la clase `SimuladorTiro` que ejecuta la integración temporal paso a paso hasta que el proyectil toca el suelo y calcula las métricas finales (altura máxima, tiempo, alcance).
3. `grafica.py`: Genera los gráficos estáticos comparativos:
 - Comparación de trayectorias (Ideal vs Real).
 - Panel cuádruple con posición, velocidad, aceleración y error a lo largo del tiempo.
- `animacion.py`: Crea una ventana interactiva donde se observa el proyectil en movimiento, su estela y un panel de telemetría (HUD) con los valores en vivo.
- `main.py` / `menu.py`: Permite al usuario elegir parámetros personalizados o ejecutar las simulaciones de prueba predeterminadas.

6. RESULTADOS Y ANÁLISIS COMPARATIVO

6.1. Parámetros de la Prueba

Para el ensayo de comparación se utilizaron los siguientes valores de prueba:

- Objeto: Esfera de 2.5 kg de masa y 15 cm de diámetro (radio 7.5 cm).
- Velocidad inicial: 70 m/s (equivalente a 252 km/h).
- Ángulo de disparo: 45°.
- Altura inicial: 0 m (nivel del suelo).
- Aire: Densidad de 1.225 kg/m³ y coeficiente de arrastre 0.47.

6.2. Tabla Comparativa de Resultados

Variable Evaluada	Modelo Ideal (Sin Aire)	Modelo Real (Con Aire - RK4)	Diferencia / Efecto
Alcance horizontal máximo	499.66 metros	278.34 metros	-44.3 % (Se reduce casi a la mitad)
Altura máxima alcanzada	124.91 metros	93.18 metros	-25.4 % (Sube menos por el frenado)
Posición X donde alcanza la altura máx.	249.83 metros	160.72 metros	El punto más alto ocurre antes
Tiempo total de vuelo	10.09 segundos	8.42 segundos	-1.67 s (Cae antes al suelo)
Tiempo hasta la altura máxima	5.05 segundos	4.02 segundos	Tarda menos tiempo en subir
Velocidad al tocar el suelo	70.00 m/s	44.78 m/s	-36.0 % (Llega con mucha menos energía)
Ángulo de impacto contra el suelo	45.0°	54.7°	Cae de forma más empinada/ vertical

6.3. Conclusiones Físicas Principales

1. Gran pérdida de alcance y energía: El rozamiento con el aire reduce el alcance del proyectil en más de un 44%. La energía cinética inicial se va disipando en forma de calor y turbulencia en el fluido circundante.
2. Pérdida de la simetría de la parábola:

- En el tiro ideal, la trayectoria es una parábola perfectamente simétrica y el punto más alto está exactamente en la mitad del camino (**50%**).
- En el tiro real, el proyectil pierde velocidad horizontal continuamente; por eso, el punto más alto se alcanza en el primer tramo (**al 57%** respecto al punto de inicio) y luego la caída es mucho más empinada y cerrada.
- Mayor ángulo y menor velocidad de impacto: Mientras que en el vacío el proyectil aterriza a la misma velocidad con la que salió (**70 m/s**) y con el mismo ángulo (**45°**), en la realidad impacta a solo **44.78 m/s** y con un ángulo de **54.7°** (más vertical).
- Eficacia del método RK4: El método numérico Runge-Kutta de 4to Orden mostró una estabilidad y exactitud total, permitiendo simular con precisión la dinámica real no lineal.

7. CÓMO EJECUTAR EL PROGRAMA

Requisitos

- Tener instalado Python 3.9 o superior.
- Librerías necesarias:

```
pip install numpy scipy matplotlib
```

Ejecución

Para iniciar el simulador, abrir una consola en la carpeta del proyecto y ejecutar:

```
python main.py
```

Archivos de salida generados:

- **trayectoria_comparativa.png**: Imagen que compara las dos curvas con las marcas de altura máxima y punto de impacto.
- **cinematica_completa.png**: Panel con 4 gráficos que detallan la evolución en el tiempo de la posición, la velocidad, la aceleración y el error numérico.
- Ventana de animación: Muestra la simulación visual con controles interactivos y panel de datos en vivo.

8. CONCLUSIÓN GENERAL

El desarrollo de este simulador permitió analizar en profundidad las diferencias cinemáticas y dinámicas entre el tiro parabólico ideal en el vacío y el modelo real bajo resistencia aerodinámica cuadrática:

1. Impacto del rozamiento del aire: Se comprobó cuantitativamente que la resistencia aerodinámica altera de forma drástica el movimiento, provocando una notable pérdida de alcance horizontal (-44.3%) y altura máxima (-25.4%), además de romper la simetría de la parábola tradicional al desplazar el punto más alto hacia la primera mitad de la trayectoria y generar una fase de caída considerablemente más empinada.
2. Efectividad del cálculo numérico: Dado que la resistencia cuadrática acopla los ejes horizontal y vertical impidiendo una solución analítica cerrada, la implementación del método Runge-Kutta de 4to Orden (RK4) junto con la interpolación de raíces para el contacto con el suelo demostró ser una solución numérica precisa, robusta y computacionalmente eficiente.
3. Aporte de la herramienta computacional: La arquitectura modular en Python combinada con salidas gráficas y una animación interactiva en tiempo real ofrece una representación visual clara de las magnitudes cinemáticas (posición, velocidad y aceleración), facilitando la comprensión del comportamiento de proyectiles en fluidos reales.